

AEGIS

Digital Public Safety Intelligence

The Complete Guide — from absolute basics

Written for someone with no computer-science background.

Every technical term is explained before it is used.

Includes presentation preparation and likely judge questions.

What's inside

1. **The big picture** — what Aegis is, and how the four parts fit together
2. **Fraud Rings** — catching mule networks, and the three Research Lab experiments
3. **Scam Detection** — messages, live calls, edge AI, and 11 languages
4. **Counterfeit Vision** — spotting fake banknotes from a photo
5. **The Dashboard and GenAI** — fusion, the money trail, and the AI Case Officer
6. **Everything else** — contracts, privacy, supply trail, deployment
7. **Achievements** — every capability, how it was built, and the bugs found along the way
8. **References** — the published research each method comes from
9. **Presentation preparation** — the questions you will be asked, and how to answer

1 · The Big Picture

Before any technical detail, understand the problem Aegis is trying to solve — because every design decision follows from it.

1.1 The real-world problem

Financial crime in India is not one crime. It is **three separate crimes that are usually run by the same people**:

Crime	What actually happens
Scam calls & messages	Someone phones you pretending to be CBI, police, or your bank. They frighten you into transferring money or sharing an OTP.
Mule account networks	The stolen money doesn't sit in one account. It is instantly split across dozens of accounts owned by ordinary people paid a small fee — "mules" — so it becomes impossible to trace.
Counterfeit currency	Fake banknotes printed and pushed into circulation, often by the same organised groups, to launder or generate cash.

Today, each of these is investigated by a **different department, using different systems, that never talk to each other**. The scam-call complaint sits in one place, the suspicious bank accounts in another, the seized fake notes in a third.

The core insight behind Aegis

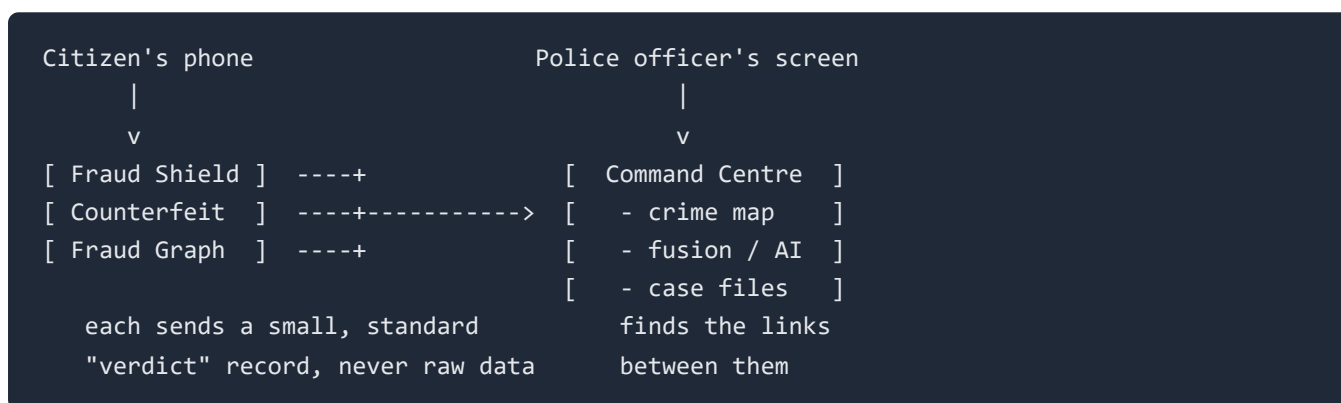
The criminals are one organisation. The response is three disconnected ones. **Aegis connects the three** — so a scam call in Jamtara, a mule ring in Alwar, and a fake note seized in Deoghar can be recognised as one operation.

1.2 The four parts

Aegis is built from four separate programs that each do one job well, plus a website that ties them together.

Part	Its one job	Technology used
Fraud Shield	Read a message or call transcript and decide: is this a scam?	Text classification (TF-IDF + Logistic Regression)
Fraud Graph	Look at bank transactions and find rings of accounts laundering money	Graph analysis + XGBoost
Counterfeit Vision	Look at a photo of a banknote and decide: real or fake?	Deep learning (EfficientNet-B0 neural network)
Command Centre	Collect everything, find connections, present it on a map	Correlation engine + Generative AI

How they connect



Analogy — a hospital

Think of three specialists: a cardiologist, a radiologist, a pathologist. Each is excellent at one thing. But nobody is looking at all three reports together to say "these three findings mean one disease." **The Command Centre is the doctor who reads all three reports.**

1.3 Some computing basics you need

These five terms appear throughout. Learn them once here.

Term	Plain meaning
Server / service	A program that runs continuously, waiting to answer questions from other programs. Aegis has four.
API	The list of questions a service will answer. Like a restaurant menu: you can order what's on it, nothing else.
Endpoint	One item on that menu. <code>/analyze</code> means "here is a message, tell me if it's a scam."
JSON	The format programs use to exchange information. Just labelled text: <code>{"verdict": "scam", "risk": 0.98}</code>
Model	A file containing what a program learned from examples. Not a brain — a large table of numbers.

Analogy — what a "model" really is

When people say "the AI", they usually imagine something thinking. It is closer to **a very long checklist with scores**. The word "free" scores +2.69 toward scam; the word "me" scores –1.59 toward safe. Add up the scores for a message; if the total crosses a line, call it a scam. That's it — no thinking involved.

2 · Fraud Rings

This is the most technically ambitious part of Aegis, and where the Research Lab lives.

2.1 The crime: what is a mule network?

When a scammer steals ₹5,00,000 from a victim, they face a problem: **a single large transfer is easy to trace and freeze**. So they do this instead:

1. The victim's money goes into **account A** (the "collector")
2. Within minutes it is split into 8 smaller transfers to accounts B, C, D...
3. Each of those splits again, and again
4. After 3–4 hops the money is withdrawn as cash or sent abroad

Those intermediate accounts belong to **real people** — often students or unemployed people paid ₹2,000 to hand over their bank card. They are called **money mules**. A group of them working together is a **mule ring**.

Why this is hard to catch

No single transaction looks suspicious. ₹48,000 moving between two ordinary accounts is completely normal. **The crime is only visible in the shape of the whole network** — dozens of accounts, all opened recently, all moving money in a tight cycle within minutes.

2.2 What is a "graph"? (the key idea)

In computing, a **graph** has nothing to do with bar charts. It means a set of *things* and the *connections* between them.

- **Node** (or vertex) = a thing. Here: **a bank account**.
- **Edge** = a connection. Here: **a transaction** from one account to another.

Analogy — a social network

On Facebook, each person is a node and each friendship is an edge. You can ask questions like "who has the most friends?" or "which people form a tight cluster who all know each other?"

Aegis asks exactly those questions about bank accounts and money. A mule ring is simply a tight cluster where the money goes round in a circle.

2.3 How Aegis finds the rings — step by step

Step 1: Measure each account (feature extraction)

For every account, the system calculates roughly 18 numbers describing its behaviour. These are called **features**. The important ones:

Feature	What it measures	Why it signals fraud
<code>throughput_ratio</code>	How much of what comes in goes straight back out	A mule keeps almost nothing — money passes through. Ratio near 1.0.
<code>burst_ratio</code>	How tightly packed the transactions are in time	Real people transact through the day; laundering happens in minutes.
<code>round_amount_ratio</code>	How often amounts are round numbers	₹50,000 exactly, repeatedly, is a script — not a human buying groceries.
<code>degree centrality</code>	How many other accounts it deals with	Collector accounts touch unusually many others.
<code>clustering_coeff</code>	Whether its partners also deal with each other	In a ring, everyone transacts with everyone — a closed loop.

Step 2: Score each account (XGBoost)

XGBoost is the learning algorithm that turns those 18 numbers into one score: the probability this account is illicit.

Analogy — how XGBoost works

Imagine a panel of hundreds of simple judges. Judge 1 only knows one rule: "if throughput is above 0.9, add suspicion." Judge 2: "if the account is under 30 days old, add a little more." Each judge alone is nearly useless. But **each new judge is specifically trained to fix the mistakes the previous ones made**, and their votes are added up. Hundreds of weak rules combine into a very accurate answer. That's "gradient boosting" — the technique behind XGBoost.

Step 3: Group the suspicious accounts into rings (Louvain)

Now we have many suspicious accounts, but which ones belong to the *same* gang? This uses **community detection** — specifically the **Louvain algorithm**.

It looks for groups of nodes that are **densely connected to each other but sparsely connected to everyone else**. In a school, it would find friendship groups. Here, it finds the ring.

```
All accounts
  |
  | keep only accounts scoring above the risk threshold
  v
High-risk accounts + the transactions between them
  |
  | Louvain community detection
  v
Ring 1 (9 accounts)   Ring 2 (8 accounts)   Ring 3 (11 accounts) ...
```

2.4 The measured results

These are real numbers from the trained model, not estimates.

Measurement	Result	What it means in plain words
ROC-AUC	0.9997	Given one fraud account and one clean account at random, the model ranks the fraud one higher 99.97% of the time. Near-perfect separation.
Average precision	0.9961	Accuracy stays high even though fraud accounts are rare.
Recall at threshold	0.973	Catches 97.3% of genuinely illicit accounts.
Rings recovered	12 / 12	Every planted ring was found. None missed.
Account precision	1.00	Zero false accusations. Every account named was genuinely part of a ring.
Account recall	0.9398	Found 94% of the members of those rings.

The honest caveat — say this before a judge asks

Those numbers come from **synthetic (simulated) data** — a generated city of accounts and transactions. Results on made-up data are always flattering, because the data was created by a program that the model can learn to reverse. This is why the next benchmark matters far more.

The real-world benchmark: Elliptic++

To prove the approach works on genuine criminal data, the same method was run against **Elliptic++** — a public, academic dataset of **real Bitcoin wallets**, where investigators have labelled which ones belong to criminal entities.

Measurement	Result	Meaning
Wallets analysed	265,354	Real wallets, real transactions
Known illicit	14,266	Labelled by researchers
ROC-AUC	0.9945	Near-identical to the synthetic result — the method transfers to real data
Average precision	0.9502	Strong on a heavily imbalanced, genuinely hard dataset

Presentation point

This is one of your strongest slides. "We didn't only test on our own simulated data — we ran the same approach on 265,000 real Bitcoin wallets from a published academic dataset and got 0.9945 AUC." That single sentence answers the "is this just a toy?" objection.

2.5 The Research Lab — three experiments

The Research Lab is a tab in the dashboard showing three deeper experiments. Each explores a question the basic system cannot answer. **Each is displayed with its honest result, including where it fell short** — which is deliberate, and worth pointing out.

Experiment 1 — Ghost Ring (cross-bank detection)

The problem: Criminals deliberately spread accounts across *different banks*. HDFC sees a quarter of the ring. ICICI sees another quarter. Neither sees a ring — each sees a few odd accounts and dismisses them. And under India's privacy law (the DPDP Act), **banks are not allowed to share customer data with each other** to compare notes.

The idea: What if banks could collaborate *without* sharing any customer data?

Analogy — describing a suspect without naming them

Four police stations each have one witness. They can't share files. But each can share a *coded description* — a string of numbers describing behaviour, from which no name or address can be recovered. A central office notices that four descriptions match the same person. **Nobody shared a file; everybody learned the truth.**

How it works technically:

1. The transaction network is split into 4 isolated "banks"
2. Each bank trains its *own* AI (a **GraphSAGE** neural network) on only its own data

3. Each publishes only an **embedding** — 64 numbers summarising an account's behaviour, from which the original data cannot be reconstructed
4. A central matcher finds embeddings from different banks that look like two ends of the same transfer
5. Those matches are stitched together, revealing the complete ring

Result	Value	Meaning
Best single bank alone	22.9%	The most any one bank could see
All four banks fused	49.4%	2.2× better than any bank alone
False-merge rate	0%	It never incorrectly joined two unrelated accounts
Differential privacy	$\epsilon = 1.0$	Strong mathematical privacy guarantee — at zero cost to accuracy

What "differential privacy" means

Before an embedding is shared, carefully calibrated random noise is added to it. Enough noise that **no individual customer's data can be extracted from it**, but not so much that the pattern is lost. ϵ (epsilon) = 1.0 is a strong guarantee by academic standards. The remarkable finding here is that switching it on **did not reduce accuracy at all** — usually privacy costs performance.

Experiment 2 — "Criminal Trains the Cop" (adversarial arms race)

The question: Criminals adapt. If our detector becomes known, they will change tactics. Does the detector survive that?

The experiment: A simulated criminal is allowed to *evolve*. It tries thousands of laundering strategies — more hops, smaller amounts, longer delays — and the ones that evade detection "survive" and are mutated further. This is a **genetic algorithm**: computational natural selection. Meanwhile the detector periodically retrains on the criminals that escaped.

Analogy — antibiotics and bacteria

Bacteria evolve resistance to a drug. Scientists develop a new drug. Bacteria adapt again. **This experiment plays out that same war inside a computer**, at high speed, to see whether the detector can keep up.

The result: Across 20 generations, the criminal repeatedly erodes the detector's accuracy — and each retraining recovers it. Recall starts at 90.6% and ends at 86.7%, moving up and down in between. That oscillation *is* the finding: a genuine arms race where neither side wins permanently.

Honest note worth making in the presentation

Getting this working required fixing four real bugs, including one where the detector kept **forgetting** older criminal tactics each time it retrained. Before the fix, recall collapsed to 9%. Saying "our first version failed and here's the bug we found" is far more credible than presenting a clean graph.

Experiment 3 — "Frequency of Fraud" (spectral analysis)

The idea: Borrow a technique from physics and signal processing. Any network has a mathematical "vibration pattern", exactly like a musical instrument has a tone. The claim being tested: **a fraud ring vibrates at a different frequency than a normal group of accounts.**

Analogy — tapping a bell

Tap a solid bell and a cracked bell. Both are bells, but they *sound* different — the crack changes how vibration travels. Aegis "taps" each community of accounts mathematically and listens. A tight artificial ring rings differently from an organic group of neighbours and shopkeepers.

The measurement is called the **Rayleigh quotient** — a single number describing how "high-frequency" (irregular) a community's structure is. A fraud ring sits measurably higher than a clean community of comparable size.

The sonification: The dashboard converts these frequencies into actual sound, so you can *hear* the difference between a clean community and a ring. It is a genuine demonstration of the underlying maths, not decoration.

The stated limitation

The panel says openly that comparing communities of *different sizes* is unreliable — a bigger community naturally scores differently for reasons unrelated to fraud. So the validated claim is narrow: **a matched pair** (a ring compared to a clean community of similar size) shows a consistent shift. Stating that limit yourself is a strength.

2.6 Where you see this in the website

Screen	What it shows
Fraud Rings tab	The list of detected rings. Click one and the money flow animates on the right — you watch the money move between accounts.
Cross-bank ring	At the top of that list. Each account is coloured by which bank holds it — the visual proof that no single bank could see the whole ring.
Inject a ring	A live demo button: it creates a brand-new laundering ring in the data and the engine catches it within seconds, in front of the audience.
Research Lab tab	The three experiments above, each with its real numbers and caveats.
Crime map	Ring locations plotted geographically alongside scams and seizures.

3 · Scam Detection (Fraud Shield)

The part a citizen actually touches — and the part with the strongest privacy story.

3.1 The crimes being detected

Scam type	How it works
Digital arrest	Someone claiming to be CBI/police says an FIR is registered against your Aadhaar. They keep you on a video call for hours, isolated from family, until you transfer money to a "safe account". India's most damaging scam by value.
KYC freeze	"Your bank KYC has expired — update within 30 minutes or your account will be blocked." The link steals your credentials.
Lottery / prize	You have won something; pay a small "processing fee" to release it.
Loan scam	Instant loan approved; pay a processing fee first.
Phishing	A link that impersonates your bank's website.

3.2 How a computer reads text — TF-IDF explained

A computer cannot read. It only does arithmetic. So the first job is turning a sentence into numbers.

Part 1: Counting words (TF — term frequency)

Count how often each word appears in the message. "Urgent" appearing twice is a stronger signal than once.

Part 2: Weighting rare words (IDF — inverse document frequency)

Some words appear in nearly every message — "the", "your", "is". They carry no information. Others are rare and therefore meaningful — "arrest", "OTP", "warrant".

Analogy — identifying a person

If I tell you a suspect "has two eyes", that's useless — everyone does. If I say they "have a scar on the left eyebrow", that's extremely useful. **IDF automatically works out which words are the scars and which are the eyes**, by measuring how rare each word is across all messages.

Part 3: Character patterns (char n-grams)

Aegis also examines **fragments of letters**, not just whole words — every run of 3, 4, and 5 characters. So "urgent", "urgnt" and "u-r-g-e-n-t" all share the fragment "urg", and a scammer's deliberate misspellings don't help them escape.

Why this matters for languages

Character fragments are **script-agnostic** — they work identically on Hindi, Tamil or English letters. This is the technical reason the same model could, in principle, be taught Indian languages directly. Remember this for section 3.7.

Part 4: The decision (Logistic Regression)

Each of the 80,011 word and character patterns has a learned **weight** — how much it pushes toward "scam" or "safe". To score a message: find which patterns it contains, add up their weights, and convert the total into a probability between 0 and 1.

Pattern	Weight	Direction
call	+3.44	strongly toward scam
free	+2.69	toward scam
www	+2.26	toward scam
me	−1.59	toward safe (personal conversation)
ok	−1.27	toward safe

3.3 Three layers, not one

The verdict does not come from the AI alone. Three layers work together:

Layer	What it does	Why it exists
1. Markers (fixed rules)	Simple pattern rules: does the message impersonate authority? demand an OTP? mention an FIR? threaten arrest?	Explainable in court. "It matched the authority-impersonation rule" is defensible; "the neural network felt uneasy" is not.
2. Classifier (the learned model)	The TF-IDF + Logistic Regression score described above	Catches scams that don't match any written rule
3. Playbooks	Known scam scripts encoded as recognisable sequences	Identifies <i>which</i> scam it is, not just that it is one

A deliberate design decision worth explaining

This module deliberately does **not** use a large language model to decide the verdict. An LLM can hallucinate — it can invent a justification that sounds convincing but is false. For evidence that may reach a courtroom, the team chose a method whose reasoning can be printed and audited line by line. **Choosing the less fashionable technology for a defensible reason is a strong point to make.**

3.4 The measured results

Measurement	Result	Meaning
ROC-AUC	0.998	Near-perfect separation of scam from legitimate
Precision (scam verdict)	0.942	When it says "scam", it's right 94% of the time
Recall (scam verdict)	0.952	It catches 95% of actual scams
Training messages	3,433	Tested on a further 1,104 it had never seen

Broken down by scam type — including the weak one

Scam family	Recall	Reading
Digital arrest	100%	Perfect — the highest-damage scam
KYC freeze	100%	Perfect
Lottery / loan / phishing	100%	Perfect
Real SMS spam (public dataset)	94.5%	Strong on genuine outside data
Investment scams	50%	The known weak spot — misses half

Say this yourself before a judge finds it

Investment-scam recall is 50%. Volunteering that — "here's where our model is weak and here's why: investment scams read like legitimate financial advice, so we need more training examples of them" — earns far more credit than being caught. It also shows you actually measured per-category instead of hiding behind one average.

3.5 Edge AI — the strongest privacy feature

This is the most important thing to understand in this section.

The problem

Normally, scanning a message means **uploading it to a server**. The citizen's private message — which may contain their name, bank, or account details — leaves their phone. For a government system, that is a serious problem.

The insight

Remember that the "AI" is just a list of 80,011 numbers. It doesn't *have* to live on a server.

Analogy — the expert's checklist

Instead of posting every letter you receive to an expert for review, **you get a copy of the expert's checklist once**, and check letters yourself at home. The letters never leave your house. That is edge AI: send the model to the phone, not the message to the model.

What was built

The entire model was exported to a file the browser downloads once — **0.9 MB compressed** — and the scoring maths was rewritten to run inside the browser.

Property	Result
Accuracy	Identical — verified to 11 decimal places against the server on 400 messages. Not an approximation: the same numbers, the same arithmetic.
Speed	0.8 milliseconds , versus a network round-trip of hundreds of milliseconds — or 50–90 seconds if the free server is asleep.
Privacy	The message never leaves the browser. Only an anonymised verdict is sent onward.
Cost	One 0.9 MB download, then cached.

The demo moment — rehearse this

Load the page, turn on aeroplane mode, paste a scam message. It still detects it.

That single action ends the privacy conversation. It isn't a claim the audience has to trust — they watch it work with no network. It also makes your demo immune to bad venue wifi.

3.6 Live Call Shield

Scam calls are more dangerous than messages, because the victim is under live psychological pressure. Live Call Shield scores the conversation **as it happens**.


```
Caller speaks
|
v
Browser converts speech to text (Chrome's built-in feature)
|
v
Running transcript scored ON THE DEVICE every ~200ms
|
v
Risk meter climbs -> at high risk, a full-screen warning interrupts
```

The escalation is the point. Watch the scripted demo call:

What the caller says	Verdict	Risk
"Good morning, calling about a courier parcel..."	legit	0.18
"I am Inspector Sharma from the CBI cyber cell"	scam	0.80
"An FIR is filed... you are under digital arrest"	scam	0.98 → interrupt
"Transfer two lakh rupees in USDT immediately"	scam	1.00

Be precise about what is private here

The **scam analysis** runs on the device — Aegis never receives the conversation. But the **speech-to-text** is Chrome's own feature, and Chrome sends the audio to Google. So the honest line is: *"Analysis runs on your device; speech-to-text is provided by your browser."* Do not claim fully private calls. If a judge checks and finds otherwise, everything else you said becomes suspect.

3.7 Multilingual support — and a subtle bug worth telling

India has 22 official languages. A scam-detection system that only works in English protects the people who least need protecting.

How it works

The model is trained in English. So for other languages, the message is **translated to English first** (using Sarvam AI, an Indian company), then scored. The app checks the *script*: Latin letters are scored on-device; Devanagari, Tamil, Bengali and others go to the server for translation.

11 languages are supported on live calls: English, Hindi, Tamil, Telugu, Bengali, Marathi, Gujarati, Kannada, Malayalam, Punjabi and Urdu.

Two real bugs that were found by testing — good material for a presentation

Bug 1 — the translator quietly reversed the meaning

The translation service was being asked to auto-detect the language. It **refused Urdu entirely**, and for Marathi and Telugu it translated "your KYC *has expired*" into "your KYC *is already completed, thank you*" — which is harmless English, and was scored as harmless. Urdu and Marathi scams were scoring 0.05 (safe).

Fix: detect the script and tell the translator explicitly which language it is. All six tested languages then scored above 0.8 (scam).

Bug 2 — "Hinglish" was completely invisible

Most real Indian scam messages are **Hindi written in English letters**: "Aapke account mein suspicious transaction detect hua hai... OTP bata dijiye." There is no Hindi character anywhere, so nothing triggered translation — and the English model has never seen "aapka" or "zaroori". It scored **0.076 (safe)**.

Fix: detect distinctive romanised Hindi words (requiring at least three, and excluding words that are also English like "main" and "hai"), then translate. It now scores 0.64 (scam), and does not false-alarm on ordinary English.

Why this belongs in your presentation

This is the single best demonstration that you tested your system rather than assuming it worked. Most teams would demo an English scam and stop. "We tested six Indian languages and found two silent failures — one where the translator inverted the meaning" shows real engineering rigour.

4 · Counterfeit Vision

Photograph a banknote; the system decides whether it is genuine — and shows you why.

4.1 The problem

Fake Indian currency (called FICN — Fake Indian Currency Notes) funds organised crime and terrorism. Detection today relies on trained eyes and machines available only in banks. A shopkeeper or constable in a small town has neither.

4.2 What is a neural network? (very basic)

The earlier modules worked on numbers and words. Images are different: a photo is millions of coloured dots, and no simple rule can turn dots into "this note is fake".

Analogy — how a network learns to see

A neural network processes an image in **layers**. The first layer detects only tiny edges — light meeting dark. The next combines edges into corners and curves. The next combines those into shapes: an eye, a letter, a pattern. The final layer combines shapes into a judgement: "this is Gandhi's portrait" or "this security thread is missing."

Nobody programmed those layers. They were **learned** from thousands of labelled examples.

4.3 Transfer learning — why this is possible at all

Training a network to see from scratch needs millions of images and enormous computing power. Aegis has a few thousand banknote photos. The solution is **transfer learning**.

Analogy — hiring an experienced photographer

You need someone to spot fake notes. You could raise a child from birth teaching them to see (impossible for you), or **hire an experienced photographer who already understands edges, textures and light**, and simply teach them the specific patterns of Indian banknotes. That's transfer learning: start from a network already trained on millions of general images, and retrain only its final decision layer.

The network used is **EfficientNet-B0** — a well-regarded architecture chosen because it is accurate but small enough to run on modest hardware.

4.4 The measured results

Measurement	Result	Meaning
Dataset	Real photographs of genuine and counterfeit notes (public Kaggle dataset), 3,508 training images	
ROC-AUC	0.9943	Excellent separation of real from fake
Accuracy	96.9%	On held-out images it never saw during training
Fake precision	0.9763	When it says "fake", it is right 97.6% of the time
Fake recall	0.9635	It catches 96.4% of counterfeits

4.5 The three-layer design

Layer	What it does	Design rule
Pre-screen (triage)	Cheap checks first: is the photo too blurry, too dark, or too small to judge? If so, ask for a rescan rather than guessing.	Never convicts. A bad photo must never become a "fake" verdict.
The CNN	The neural network gives the genuine/fake verdict	Sole authority on the verdict
Serial + vision checks	Checks the serial number format, whether that serial has been seen before, and whether the portrait/header are correct	Can downgrade "genuine" to "needs manual check" — never the reverse

A real bug worth telling — it shows judgement

An earlier version let the cheap pre-screen layer declare notes fake. Its thresholds had been tuned on *computer-generated* images, and on real phone photos it misfired constantly: dim lighting reduced colour saturation, and ordinary photo compression reduced texture — so a **genuine** note tripped two checks at once and was branded counterfeit. Almost every real note was being called fake.

The fix: the pre-screen became advisory only. It still records what it measured, but only the neural network — trained on real photographs — can decide.

4.6 The heatmap (Grad-CAM) — explainability

A verdict alone is not enough for an officer. They need to know *why*. The system produces a **heatmap** overlaid on the note, colouring the regions that most influenced the decision.

Analogy — a teacher marking an essay

A grade of 6/10 with no marks on the page is useless. A teacher who **underlines the specific sentences** that lost marks is teaching. Grad-CAM underlines the parts of the note that drove the verdict.

An engineering compromise worth knowing

The standard heatmap technique needs a "backward pass", which roughly triples memory use and crashed the free-tier server mid-scan. So on low-memory machines the system switches to **Eigen-CAM**, which produces a heatmap using only the forward pass. **The verdict is byte-identical either way** — only how the picture is computed changes.

4.7 The serial-number registry

A clever extra check. Every genuine note has a unique serial number. Counterfeiters typically print **one real serial onto thousands of notes**. So Aegis remembers every serial it has seen, and if the same serial appears in two different districts, that is strong evidence of a printing run — a link the image alone could never provide.

5 · The Dashboard and Generative AI

This is where the three modules stop being separate tools and become one intelligence picture.

5.1 Fusion — finding connections

The **correlator** is a deterministic program (no AI) that looks for real, checkable links between events from different modules.

The strongest link: the money trail

This is the single best result in Aegis, and the one to demonstrate:

1. A victim reports a scam call, and reports that they paid **₹49,999**
2. The fraud graph independently knows every transaction between mule accounts
3. The correlator looks for an incoming payment of that **same amount** (within 1%), arriving **shortly after** the call
4. It finds ₹49,999 landing in account `acc_02033`, part of detected ring `ring_05`

Why this is powerful

Two completely independent systems — one reading text, one reading bank transactions — **independently point at the same crime**. The dashboard shows this as an orange "money trail traced" card: *the money you lost went here*. That is what no single department can currently produce.

Other links the correlator finds

- **Scam ↔ ring**: a scam and a mule ring in the same district within a time window
- **Scam ↔ counterfeit**: both crime types clustering in one place
- **Plate families**: fake notes from different districts sharing the same printing defects — the same physical printing plate
- **Scam campaigns**: different victims receiving near-identical scripts, or the same callback number — one operation, not isolated incidents

5.2 Where Generative AI is used — and where it is deliberately not

This distinction is the most important thing in this section.

Task	Uses an LLM?	Reason
Deciding if a message is a scam	No	Must be auditable and court-defensible
Deciding if a note is fake	No	Trained neural network, measurable accuracy
Deciding which accounts form a ring	No	Deterministic graph algorithms
Finding correlations between events	No	Exact arithmetic on amounts and timestamps
<i>Explaining</i> those findings in English	Yes	Writing readable prose is exactly what LLMs are good at
<i>Writing</i> the case-file narrative	Yes	Same — it narrates facts it is given, and invents none

The principle, in one line

The deterministic engines decide. The AI only explains. Remove every LLM from Aegis and every verdict stays exactly the same — you simply lose the readable summaries. That is a deliberate architecture, and a strong answer to "how do you stop AI hallucination?"

5.3 The fallback chain — why the demo cannot die

AI services fail: they go down, hit rate limits, or get blocked. So every AI feature tries providers in order, and ends with a guaranteed non-AI option:

```
Claude -> Groq -> Gemini -> Groq (spare) -> TEMPLATE
                                     |
                                     a plain-English summary written by
                                     ordinary code — always works, needs
                                     no internet, no key, and cannot fail
```

The **template** at the end is the key design decision. Even with zero API keys and no internet, every panel still produces a sensible summary of the same facts. **The demo cannot go blank.**

Real incidents from this project

Two genuine failures were found and fixed: **Google retired the Gemini model** being used (every call returned "no longer available"), and the **timeouts were shorter than the AI needed** to write its answer — so every provider was cut off mid-sentence and silently fell through to the template. Both are good examples of why the fallback exists.

5.4 The AI Case Officer

One click on a district produces a complete case brief: a summary, a chronological timeline, a *hedged* hypothesis, and recommended next steps — each citing the specific evidence it came from.

It works in two strictly separated stages:

1. **Gather evidence** (deterministic): collect every scam, ring, seizure and correlation for that district into a dossier of established facts
2. **Write the brief** (AI): the LLM reads *only* that dossier and writes it up. It is forbidden from referring to anything outside it.

Note the careful language

The brief always hedges — "consistent with", "suggests", "worth checking" — and never names individuals or asserts guilt. This matters: a system that produces confident accusations about citizens is dangerous. A system that produces *investigative leads* is useful.

5.5 The dashboard screens

Tab	What it shows
Live Map	Every detection plotted geographically. Hotspots where crime types overlap are highlighted.
Modules	Health of each detection service and links to the citizen-facing tools
Fraud Rings	Detected rings, including the cross-bank ring, with animated money flow
Alerts & Analytics	Live alerts, scam campaigns, plate families, and the orange money-trail card
Disrupt	Recommended actions — freeze this account, block this number — derived from the findings
Metrics	The model card: measured accuracy of every model, including weaknesses
Research Lab	The three experiments from section 2.5

6 · Everything Else

6.1 Contracts — how the parts stay compatible

Four programs written by different people must exchange data without breaking each other. Aegis uses **contracts**: strict written definitions of exactly what each message must contain.

Analogy — a standard form

A hospital admission form has fixed fields: name, age, blood group. Anyone can fill it in, and any department can read it, because **the shape is agreed in advance**. If a field is missing, the form is rejected at the desk — not three departments later.

Every message is validated against its contract on arrival. Malformed data is rejected immediately rather than corrupting the dashboard.

6.2 Privacy — the audit

A full privacy audit was carried out across every module, tracing every piece of data that leaves the system. It found **15 exposures**, each recorded with the exact file and line, and ranked by severity.

Finding	Issue	Status
P1	No authentication on any endpoint — stored messages were publicly readable	Fixed
P2	Real phone numbers were being sent to AI providers abroad	Fixed — replaced with irreversible hashes
P3	Full message text sent to an AI provider	Fixed — only a redacted excerpt now
P6	Message text stored on the server	Fixed for English — scored on the device instead
P7	Call audio reaches Google via the browser	Partial — documented honestly
P10/P11	Map searches and map tiles reveal which district police are investigating	Open — documented

Presentation angle

Having a written privacy audit that lists your *own* unfixed problems is unusual and impressive. If a judge asks about privacy, showing that document is a stronger answer than any verbal claim. The two findings that leak the *investigator's* activity (P10/P11) are worth mentioning — most teams never consider that direction.

6.3 Supply Trail

When counterfeit notes are seized, where did they enter the country? Supply Trail uses **Dijkstra's algorithm** — the same shortest-path method used by map apps — over India's real rail, road and air corridors, to compute plausible routes from a likely origin to the seizure location.

Stated honestly in the product

A banknote carries no origin label. The route is a **hypothesis for investigators**, not evidence. The confidence score is capped below 1.0 by design, and the interface says so explicitly.

6.4 Deployment — how it runs on the internet

Piece	Where it runs	Why
Four services	Render (free tier)	Runs Python programs continuously
Dashboard	Vercel	Built for websites, fast worldwide

The free-tier reality — worth knowing before demo day

Free services **sleep after 15 minutes** of no traffic and take 50–90 seconds to wake. A sleeping service looks identical to a broken one.

Always open all four service links 15 minutes before presenting. And note the safety net: because scam detection runs in the browser, that demo **still works even if every server is asleep**.

7 · Achievements — what was built, and how

Every capability below, with the method that produced it and the measured result. This is the section to revise before a technical question.

7.1 Detection capabilities

Achievement	How it was done	Measured result
Scam text detection	TF-IDF over word 1–2-grams <i>and</i> character 3–5-grams, fed to Logistic Regression; thresholds chosen from the precision–recall curve rather than a default 0.5	ROC-AUC 0.998 ; precision 0.942, recall 0.952 on 1,104 unseen messages
Explainable markers	8 hand-written pattern rules (authority impersonation, fake FIR, OTP request, urgency, isolation) run alongside the model	Every verdict carries named reasons, printable for court
Scam-type identification	Encoded playbooks matching known scam scripts	Names the scam (digital arrest, KYC, lottery), not just "suspicious"
Mule-ring detection	18 graph features per account → XGBoost → Louvain community detection over high-risk accounts only	ROC-AUC 0.9997 ; 12/12 rings recovered; account precision 1.00
Real-world validation	Same pipeline re-run on the public Elliptic++ Bitcoin dataset (55 official features, deduplicated to latest snapshot)	265,354 wallets, 0.9945 AUC — proves transfer beyond synthetic data
Counterfeit detection	EfficientNet-B0 with frozen ImageNet backbone, retrained head only (the correct recipe for a few-thousand-image dataset)	ROC-AUC 0.9943 , accuracy 96.9%, fake recall 96.4%
Visual explanation	Grad-CAM heatmap over the note, with an Eigen-CAM fallback for low-memory hosts	Officer sees <i>which regions</i> drove the verdict; verdict identical either way
Serial-reuse detection	Every scanned serial recorded in a registry (MongoDB, with file fallback)	Same serial in two districts implies one printing run

7.2 Research achievements

Achievement	How it was done	Measured result
Cross-bank detection without sharing data	Graph split into 4 isolated silos; per-silo GraphSAGE; only 64-dim boundary embeddings published; Hungarian assignment matches them; Leiden clustering on the fused graph	Fused 49.4% vs best single bank 22.9% = 2.2× , with a 0% false-merge rate
Formal privacy on that exchange	Gaussian mechanism applied to every embedding before publication; per-bank salted BLAKE2b pseudonyms replace account ids	($\epsilon=1.0$, $\delta=1e-5$)-DP at zero accuracy cost — identical recall gap with DP on and off
Adversarial robustness testing	Genetic algorithm (DEAP) evolves laundering strategies against the live detector; detector retrains on escapees every 5 generations	Genuine see-saw over 20 generations; recall 0.906 $\rightarrow$ 0.867 with recovery after each retrain
Spectral fraud signature	Normalised graph Laplacian per community; Rayleigh quotient measures high-frequency energy; matched-pair comparison by size	Ring communities sit measurably higher-frequency than size-matched clean ones
Sonification	Eigenvalues mapped to pitch (200 Hz–4 kHz), spectral energy to amplitude, synthesised live in the browser via Web Audio	The structural difference is <i>audible</i> — no audio files needed
Supply-trail inference	Dijkstra + Yen k-shortest-paths over real Indian rail/road/air corridors; plausibility = distance decay + mode risk + FIR corroboration	Ranked entry routes with confidence capped below 1.0 by design

7.3 Privacy and engineering achievements

Achievement	How it was done	Result
On-device scam scoring	Model exported to JSON (vocabulary + IDF + coefficients); TF-IDF, char n-grams, accent stripping and sigmoid reimplemented in JavaScript; a parity test fails the build if browser and server ever disagree	Identical to 11 decimal places on 400 messages; 0.8 ms per message; 0.9 MB gzipped
Full privacy audit	Traced every outbound call, persisted field and contract boundary across all modules; each finding cited to file and line	15 findings ranked; both criticals fixed; audit published in the repo
PII redaction at source	Keyed BLAKE2b phone pseudonyms plus regex masking of digits, emails, links and UPI handles, applied in one module so no consumer can forget	No phone digits or message text reach any AI provider or public endpoint
Write authentication	Shared-secret header guard on mutating endpoints, opt-in via env var so a demo cannot break; <code>/health</code> reports whether it is active	Crime map can no longer be poisoned or wiped by anyone with the URL
Multilingual coverage	Script detection maps to an explicit Sarvam source language; Hinglish detected by distinctive romanised tokens	11 languages on live calls; 6 tested languages all score above 0.8
Never-fail AI	Provider chain Claude → Groq → Gemini → spares → deterministic template floor	Every AI panel produces output with zero API keys and no internet
Contract enforcement	JSON Schema per signal type, validated on ingest	Malformed data rejected at the door, not three modules later
Memory optimisation	Heavy libraries made lazy; startup serves the shipped artifact instead of recomputing	Fraud-graph 300 MB → 48 MB at rest, ending the OOM restarts

7.4 Bugs found and fixed — the engineering story

Each of these was found by *measuring*, not by assuming. They matter because "how did you validate it?" is a common question.

Bug	Symptom	Root cause and fix
Unfair Ghost Ring scoring	Fusion appeared to <i>lose</i> to a single bank (49% vs 73%)	Each bank was graded only on accounts it could already see, fusion on the whole network — different denominators. Scored fairly: 49.4% vs 22.9%, a 2.2× win.
Detector amnesia	Arms-race recall collapsed to 9%	Evader memory was wiped after each retrain, so the detector forgot every earlier tactic. Memory now persists.
Identical evader IDs	Detector retrained on 1 example instead of 5	Every injected ring generated the same account IDs, so they collapsed on merge. Namespaced per ring.
Frozen detection threshold	Retraining changed nothing	Ring linking cut at a hardcoded 0.5 while the model calibrated to 0.0049, so retraining could not affect detection. On one evader: 7/51 caught vs 46/51.
Pre-screen convicting real notes	Almost every genuine note called fake	Thresholds tuned on synthetic renders misfired on real photos (dim light, JPEG compression). Pre-screen made advisory only.
Translator inverting meaning	Urdu and Marathi scams scored 0.05 (safe)	Auto-detect refused Urdu and rendered "KYC has expired" as "KYC is already completed". Explicit source language fixed all six languages.
Hinglish invisible	Romanised Hindi scam scored 0.076 (safe)	Latin script triggered no translation and the English model knows no "aapka". Token-based detection added.
Dead Gemini model	All AI summaries silently fell back to template	Google retired <code>gemini-2.0-flash</code> ; every call returned 404. Timeouts were also shorter than generation time. Both fixed; 28 s → 4.9 s.
Health probe too fast	Dashboard showed modules offline while they served fine	1.5 s timeout tuned for localhost; real cross-host probes take 2.6–3.5 s. Raised and made concurrent.

8 · References

The published methods Aegis builds on. Only techniques actually implemented in the codebase are listed.

8.1 Graph machine learning

Method	Reference	Where used
GraphSAGE inductive node embeddings	Hamilton, Ying & Leskovec, <i>Inductive Representation Learning on Large Graphs</i> , NeurIPS 2017	Ghost Ring — each bank trains a local model and publishes boundary embeddings
BWGNN Beta wavelet graph neural network	Tang et al., 2022 — <i>Rethinking Graph Neural Networks for Anomaly Detection</i> , ICML (cited directly in <code>spectral.py</code>)	Spectral module — band-pass filters isolating frequency bands of the Laplacian spectrum
Louvain community detection	Blondel, Guillaume, Lambiotte & Lefebvre, <i>Fast unfolding of communities in large networks</i> , J. Stat. Mech. 2008	Grouping high-risk accounts into candidate rings
Leiden improved community detection	Traag, Waltman & van Eck, <i>From Louvain to Leiden: guaranteeing well-connected communities</i> , Scientific Reports 2019	Ghost Ring — clustering the fused cross-bank graph
Spectral graph theory Laplacian, Rayleigh quotient	Chung, <i>Spectral Graph Theory</i> , AMS 1997	"Frequency of Fraud" — measuring a community's structural frequency

8.2 Machine learning

Method	Reference	Where used
XGBoost gradient-boosted trees	Chen & Guestrin, <i>XGBoost: A Scalable Tree Boosting System</i> , KDD 2016	Scoring accounts for ring membership
EfficientNet compound-scaled CNN	Tan & Le, <i>EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks</i> , ICML 2019	Counterfeit Vision backbone (B0 variant)
Grad-CAM visual explanation	Selvaraju et al., <i>Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization</i> , ICCV 2017	Heatmap showing which note regions drove the verdict
Eigen-CAM forward-only explanation	Muhammad & Yeasin, <i>Eigen-CAM: Class Activation Map using Principal Components</i> , IJCNN 2020	Low-memory heatmap fallback
TF-IDF term weighting	Spärck Jones, <i>A Statistical Interpretation of Term Specificity and Its Application in Retrieval</i> , J. Documentation 1972	Turning scam messages into numbers

8.3 Privacy

Method	Reference	Where used
Differential privacy the formal definition	Dwork, McSherry, Nissim & Smith, <i>Calibrating Noise to Sensitivity in Private Data Analysis</i> , TCC 2006	The privacy guarantee on published embeddings
Gaussian mechanism noise calibration	Dwork & Roth, <i>The Algorithmic Foundations of Differential Privacy</i> , 2014 — $\sigma = \text{sensitivity} \times \sqrt{2 \ln(1.25/\delta)}$ / epsilon	Exactly this formula is implemented in <code>ghost_ring_privacy.py</code>
Federated learning the concept	McMahan et al., <i>Communication-Efficient Learning of Deep Networks from Decentralized Data</i> , AISTATS 2017	Conceptual basis for Ghost Ring's per-bank training
BLAKE2 keyed hashing	Aumasson, Neves, Wilcox-O'Hearn & Winnerlein, <i>BLAKE2: simpler, smaller, fast as MD5</i> , ACNS 2013	Salted phone and account pseudonyms

8.4 Algorithms

Method	Reference	Where used
Dijkstra shortest path	Dijkstra, <i>A Note on Two Problems in Connexion with Graphs</i> , Numerische Mathematik 1959	Supply-trail routing over transport corridors
Yen's algorithm k-shortest paths	Yen, <i>Finding the K Shortest Loopless Paths in a Network</i> , Management Science 1971	Alternative entry routes, not just the single best
Hungarian algorithm optimal assignment	Kuhn, <i>The Hungarian Method for the Assignment Problem</i> , Naval Research Logistics 1955	Matching boundary embeddings across banks
Genetic algorithms	Holland, <i>Adaptation in Natural and Artificial Systems</i> , 1975 — implemented via the DEAP library	Evolving criminal strategies in the arms race
Damerau–Levenshtein edit distance	Damerau, <i>A technique for computer detection and correction of spelling errors</i> , CACM 1964	Matching misspelled district names in search

8.5 Datasets

Dataset	Source	Used for
Elliptic++	Elmougy & Liu, <i>Demystifying Fraudulent Transactions and Illicit Nodes in the Bitcoin Network</i> , KDD 2023 — 265,354 wallets, 55 official features	Real-world benchmark for fraud-ring detection (0.9945 AUC)
SMS Spam Collection	Almeida, Gómez Hidalgo & Yamakami, ACM DOCENG 2011 — UCI ML Repository, 5,574 labelled SMS	Real-message component of scam training data
Indian currency real vs fake	Public Kaggle dataset (preetrunk) — real photographs of genuine and counterfeit notes, by denomination	Training the counterfeit CNN

8.6 Services and libraries

Component	Purpose
Sarvam AI	Indian translation service covering all 22 scheduled languages, used to normalise non-English messages to English before scoring. An Indian provider, which matters for data-residency arguments.
PyTorch / PyTorch Geometric	Neural networks (EfficientNet, GraphSAGE)
scikit-learn	TF-IDF and Logistic Regression
NetworkX	Graph construction and feature computation
FastAPI	All four service APIs
Next.js / React	The dashboard
MapLibre	The crime map

How to use this section in a presentation

You will not recite references. But if a judge asks *"is this your own idea or published work?"*, the strong answer is: **"The methods are published and peer-reviewed — GraphSAGE, BWGNN, the Gaussian mechanism for differential privacy. What is ours is the combination: applying federated embeddings plus differential privacy to cross-bank mule detection, and fusing three crime types into one intelligence picture."** Standing on established research is a strength, not a weakness.

9 · Presentation Preparation

9.1 A suggested 7-minute structure

Time	Section	Content
0:00–1:00	The problem	Three crimes, one criminal organisation, three disconnected departments
1:00–2:30	Scam detection	Live demo. Then the aeroplane-mode moment.
2:30–4:00	Fraud rings	Inject a ring live, watch it caught. Show the cross-bank ring.
4:00–5:00	Counterfeit	Scan a note, show the heatmap
5:00–6:30	The fusion moment	The orange money trail: the victim's ₹49,999 traced into a specific mule account
6:30–7:00	Close	Privacy: nothing leaves the device. Real benchmark: 265,000 Bitcoin wallets.

9.2 The questions you will be asked

"Does the citizen's message go to your server?"

"No. The classifier runs in the browser — the same model, verified score-for-score against the server version. Watch: aeroplane mode, still works."

"What do you send to your servers, then?"

"Verdict, scam type, district, timestamp. Not the message. That's everything the national crime map needs, and nothing more."

"Doesn't your AI see the message?"

"No. The AI only receives tool results and a redacted excerpt — digits, links and IDs masked. Phone numbers are irreversible hashes, never raw."

"How do you prevent AI hallucination?"

"The AI never decides anything. Every verdict comes from a deterministic model or algorithm. The AI only writes the English explanation of facts it is handed. Remove it entirely and every verdict is unchanged."

"Is this just trained on your own made-up data?"

"The fraud graph was also benchmarked on Elliptic++ — 265,354 real Bitcoin wallets from a published academic dataset — at 0.9945 AUC. The counterfeit model is trained on real photographs, and the scam model includes a public SMS spam corpus."

"What about languages other than English?"

"Eleven languages on live calls. English is scored on the device; other languages are translated and scored on our server. We also detect Hinglish — Hindi written in English letters — which is how most real Indian scam messages are actually written."

"What doesn't work? / What are the limitations?"

Answer this one honestly and specifically — it is often the question that decides the result.

"Three things. Investment scams: we catch about half — they read like legitimate financial advice. Live call speech-to-text goes through the browser, so audio reaches Google — production would use on-device speech or telecom integration. And the fraud-ring numbers on synthetic data are flattering, which is why we also benchmarked on real Bitcoin data."

"Could this actually be deployed?"

"For SMS, yes — a mobile app can read messages on Android, and iOS has an on-device SMS filter API designed for exactly this. For calls, no app on iOS or Android is permitted to tap call audio, so production would require telecom-operator integration. That's a policy constraint, not a technical one."

9.3 Three things to avoid saying

Don't say	Say instead
"Everything runs on-device"	"Scam analysis runs on-device; speech-to-text is the browser's service"
"Our AI decides if it's a scam"	"A deterministic model decides; the AI writes the explanation"
"100% accurate"	Quote the actual number, and the category where it's weakest

9.4 The five numbers to memorise

Number	What it is
265,354	Real Bitcoin wallets benchmarked — proof it isn't only synthetic data
0.9945	AUC on that real dataset
2.2×	How much more of a cross-bank ring is visible when four banks fuse privately vs the best single bank
0.8 ms	On-device scam scoring time — faster than any network call
₹49,999	The traced money trail: a victim's payment followed into a specific mule account

A closing thought for your presentation

The most persuasive thing about Aegis is not any single accuracy figure. It is that the system **states its own limitations on screen** — the failed fusion result, the 50% investment-scam recall, the spectral method's caveat, the privacy audit listing unfixed items.

Anyone can show a demo that works. Showing that you know exactly where it doesn't — and measured it yourself — is what distinguishes engineering from a prototype.